Unit 8: Routing

Introduction

Routing is the act of moving information across an inter-network from a source to a destination. Along the way, at least one intermediate node typically is encountered. It's also referred to as the process of choosing a path over which to send the packets. The routing algorithm is the part of the network layer software responsible for deciding which output line an incoming packet should be transmitted on, i.e. what should be the next intermediate node for the packet.

Routing protocols use metrics to evaluate what path will be the best for a packet to travel. ***A metric is a standard of measurement; such as path bandwidth, reliability, delay, current load on that path*** etc. used by routing algorithms to determine the optimal path to a destination. To aid the process of path determination, routing algorithms initialize and maintain routing tables, which contain route information. Route information varies depending on the routing algorithm used.

Desirable properties of routing are as follows:

- ✓ **Correctness and simplicity:** The packets are to be correctly delivered. Simpler the routing algorithm, it is better.
- ✓ **Robustness:** Ability of the network to deliver packets via some route even in the face of failures.
- ✓ **Stability:** The algorithm should converge to equilibrium fast in the face of changing conditions in the network.
- ✓ **Fairness and optimality:** obvious requirements, but conflicting.
- ✓ **Efficiency:** Minimum overhead

While designing a routing protocol it is necessary to take into account the following design parameters:

- ✓ **Performance Criteria:** Number of hops, Cost, Delay, Throughput, etc.
- ✓ **Decision Time:** Per packet basis (Datagram) or per session (Virtual-circuit) basis
- ✓ **Decision Place:** Each node (distributed), Central node (centralized), Originated node (source)
- ✓ **Network Information Source:** None, Local, Adjacent node, Nodes along route, All nodes
- ✓ **Network Information Update Timing:** Continuous, Periodic, Major load change, Topology change

Classification of Routing Algorithms

Routing algorithms can be classified based on the following criteria:

1. Static versus Adaptive (Dynamic)
2. Single-path versus multi-path
3. Intra-domain versus inter-domain
4. Flat versus hierarchical

Static versus Adaptive

This category is based on how and when the routing tables are set-up and how they can be modified, if at all. Adaptive routing is also referred as **dynamic routing** and Non- adaptive is also known as **static routing** algorithms. *Static routing algorithms* are hardly algorithms at all; the table mappings are established by the network administrator before the beginning of routing. These mappings do not change unless the network administrator alters them. ***Dynamic routing algorithms***, which adjust to changing network circumstances by analyzing incoming routing update messages. If the message indicates that a network change has occurred, the routing software recalculates routes and sends out new routing

update messages. These messages permeate the network, stimulating routers to rerun their algorithms and change their routing tables accordingly. Dynamic routing algorithms can be supplemented with static routes where appropriate.

Single-Path versus Multi-path

This division is based upon the number of paths a router stores for a single destination. Single path algorithms are where only a single path (or rather single next hop) is stored in the routing table. Some sophisticated routing protocols support multiple paths to the same destination; these are known as multi-path algorithms. Unlike single-path algorithms, these multipath algorithms permit traffic multiplexing over multiple lines. The advantages of multipath algorithms are obvious: They can provide substantially better throughput and reliability. This is generally called load sharing.

Intra-domain versus Inter-domain

Some routing algorithms work only within domains; others work within and between domains. The nature of these two algorithm types is different. It stands to reason, therefore, that an optimal intra-domain-routing algorithm would not necessarily be an optimal inter-domain-routing algorithm.

Flat Versus Hierarchical

Some routing algorithms operate in a flat space, while others use routing hierarchies. In a *flat routing system*, the routers are peers of all others. In a hierarchical routing system, some routers form what amounts to a routing backbone. Packets from non-backbone routers travel to the backbone routers, where they are sent through the backbone until they reach the general area of the destination. At this point, they travel from the last backbone router through one or more non-backbone routers to the final destination.

Routing systems often designate logical groups of nodes, called domains, autonomous systems, or areas. In *hierarchical systems*, some routers in a domain can communicate with routers in other domains, while others can communicate only with routers within their domain. In very large networks, additional hierarchical levels may exist, with routers at the highest hierarchical level forming the routing backbone.

Routing Algorithm Metrics

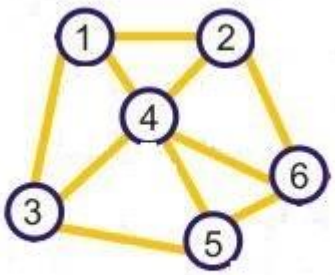
Routing algorithms have used many different metrics to determine the best route. Sophisticated routing algorithms can base route selection on multiple metrics, combining them in a single (hybrid) metric. All the following metrics have been used:

- Path length
- Delay
- Bandwidth
- Load
- Communication cost
- Reliability

Fixed or Static Routing

In fixed routing a route is selected for each source-destination pair of nodes in the network. The routes are fixed; they may only change if there is a change in the topology of the network. A central routing matrix is created based on least-cost path, which is stored at a network control center. The matrix shows, for each source-destination pair of nodes, the identity of the next node on the route. Figure 1 (a) shows a simple packet switching network with six nodes (routers), and Figure 1 (b) shows

the central routing table created based on least-cost path algorithm. Figure 2 (b) shows the routing tables that can be distributed in different nodes of the network.



Central Routing Directory

From node \ To node	1	2	3	4	5	6
1	-	2	3	2	2	2
2	1	-	1	6	6	6
3	1	1	-	4	5	1
4	6	6	3	-	5	6
5	4	4	3	4	-	4
6	2	2	2	4	4	-

Figures: 1 (a) A simple packet switching network with six nodes (routers), (b) The central routing table created based on least-cost path

Node 1 Directory

Destination	Next Node
2	2
3	3
4	2
5	2
6	2

Node 2 Directory

Destination	Next Node
1	1
3	1
4	6
5	6
6	6

Node 3 Directory

Destination	Next Node
1	1
2	1
4	4
5	5
6	1

Node 4 Directory

Destination	Next Node
1	6
2	6
3	3
5	5
6	6

Node 5 Directory

Destination	Next Node
1	4
2	4
3	3
4	4
6	4

Node 6 Directory

Destination	Next Node
1	2
2	2
3	2
4	4
5	4

Figure 2: Routing tables that can be stored in different nodes of the network.

Flooding

Flooding requires no network information whatsoever. Every incoming packet to a node is sent out on every outgoing line except the one it arrived on. All possible routes between source and destination are tried. A packet will always get through if a path exists. As all routes are tried, at least one packet will pass through the shortest route. All nodes, directly or indirectly connected, are visited. Main limitation flooding is that it generates vast number of duplicate packets. It is necessary to use suitable damping mechanism to overcome this limitation. One simple is to use *hop-count*; a hop counter may be contained in the packet header, which is decremented at each hop, with the packet being discarded when the counter becomes zero. The sender initializes the hop counter. If no

estimate is known, it is set to the full diameter of the subnet. Another approach is keep track of packets, which are responsible for flooding using a sequence number and avoid sending them out a second time. A variation, which is slightly more practical, is *selective flooding*. The routers do not send every incoming packet out on every line, only on those lines that go in approximately in the direction of destination. Some of the important utilities of flooding are:

- ✓ Flooding is highly robust, and could be used to send emergency messages (e.g., military applications).
- ✓ It may be used to initially set up the route in a virtual circuit.
- ✓ Flooding always chooses the shortest path, since it explores every possible path in parallel.
- ✓ Can be useful for the dissemination of important information to all nodes (e.g., routing information).

Static Algorithms

Shortest Path Routing (Dijkstra's Algorithm):

The first five steps used in computing the shortest path from A to D. The arrows indicate the working node. Assume the "distance" (measured in hops, or physical distance, or time) on each link is known. Each node is labeled with its distance along the best known path. At first there is no known path and all nodes are labeled with o. The source A is the first permanent node; so, the circle is filled in. Then all nearby nodes are labeled with their distance from A and the nearest node is made permanent (in this case B). All nodes near the permanent are searched and if their distances can be reduced, they are. Then all tentatively labeled nodes are searched for the minimum distance to A, and the one with the minimum distance becomes the new permanent node.

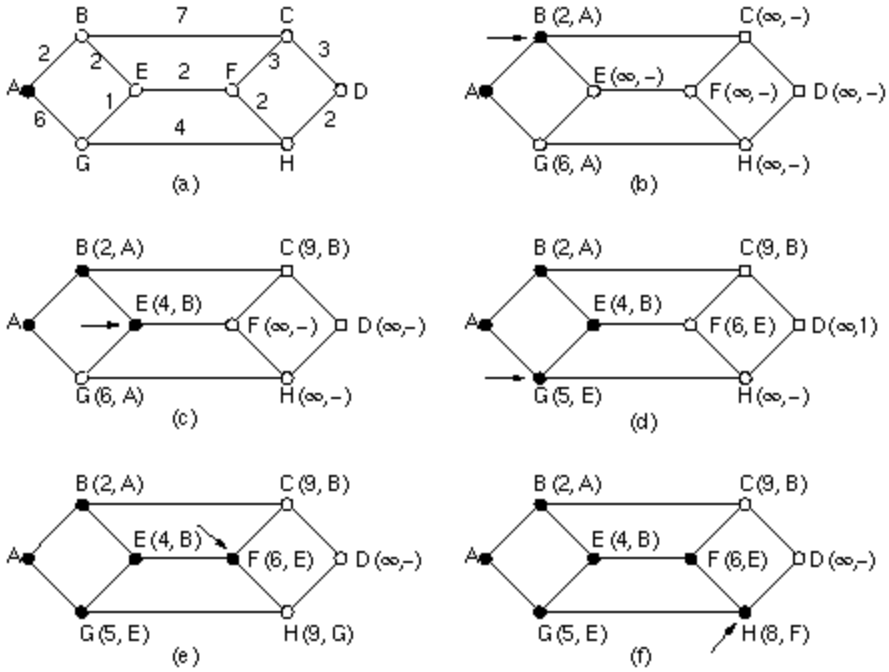


Figure: Demonstration of Dijkstra's Algorithm for finding shortest path (A to D)

Dynamic Routing Algorithms

1. Distance Vector Routing

In a large network, it is not desirable to have complete information on every router in the network. In distance vector routing, each node shares its routing table with its immediate neighbors periodically and when there is a change in the network topology. In this routing the routing table of a node is updated using the routing table of its adjacent nodes only. It was first used in ARPANET, on the Internet (as RIP). Each router contains a routing table with: (1) the distance to each router; (2) the preferred outgoing line. The metric can be delay time, number of hops, physical distance, etc. It must also know the distance to every neighbor. In Every T msec, it exchanges information with its neighbors, finds which delays are minimal and updates its table. The algorithm used in this routing is called ***Bellman Ford Algorithm.***

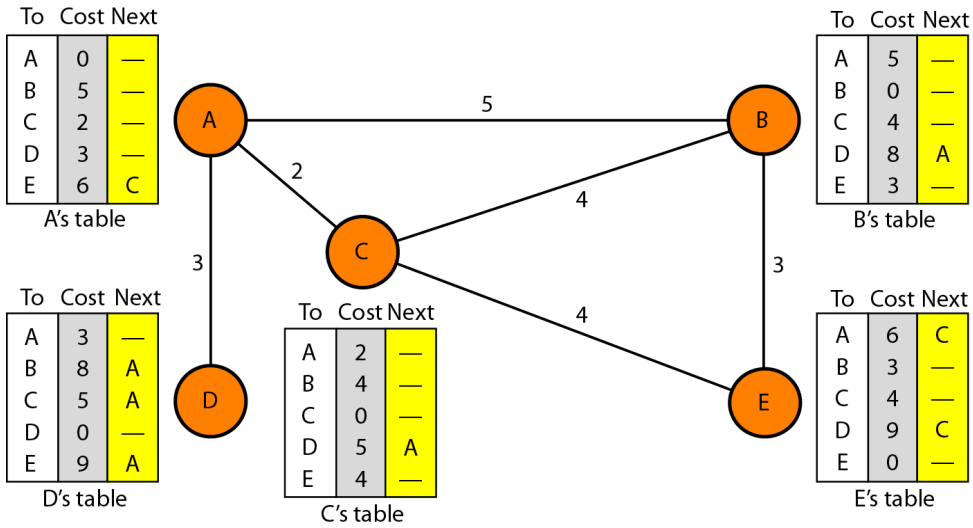


Figure: Routing Table of nodes in a network

Now, updating routing table of A based on routing table of C:

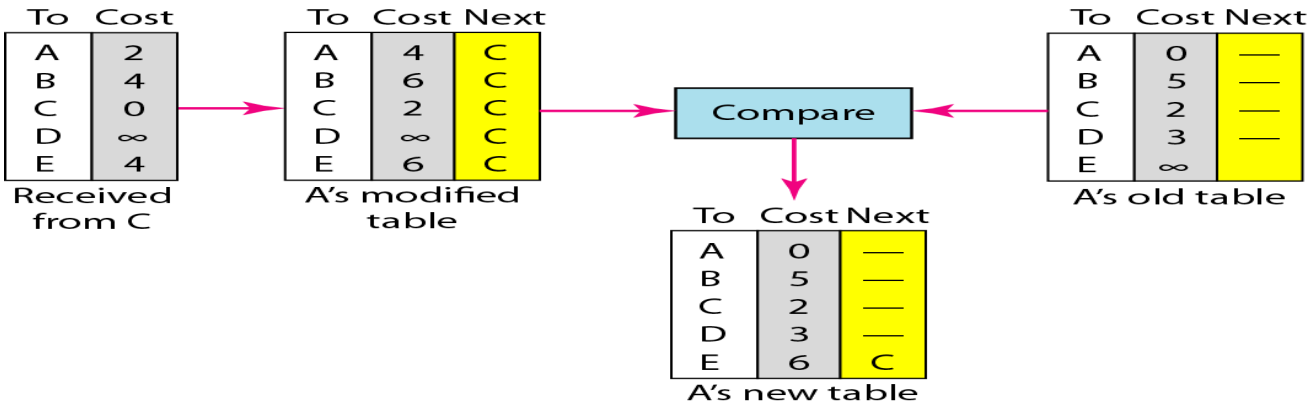


Figure: Updated routing table of A based on adjacent node C routing table

2.Link State Routing

This overcomes two disadvantages of distance vector routing, link state routing has been used in the Internet since 1979 and is now the preferred approach.

It is based on a five step algorithm.

- Learn about neighbors.
- Measure line cost.

- Build link state packets.
- Distribute link state packets.
- Compute new routes.

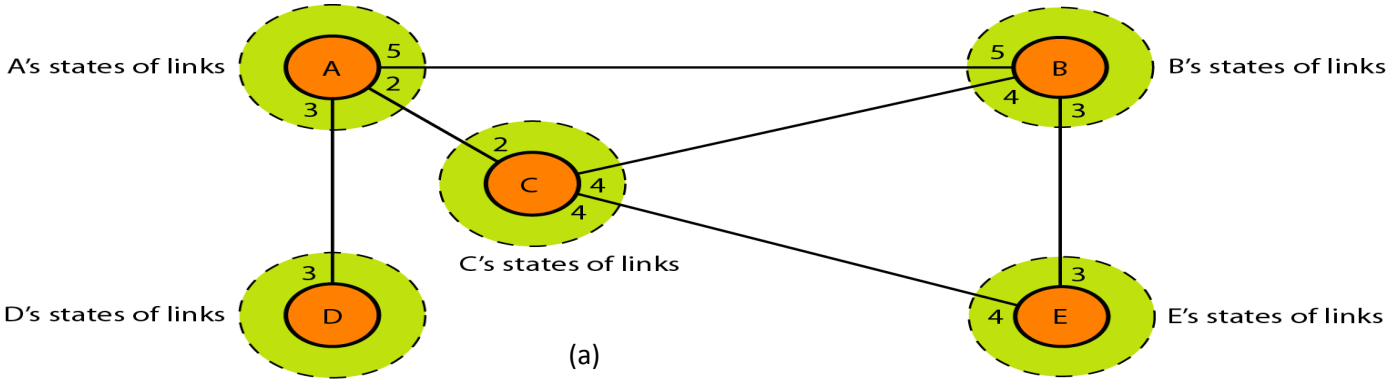
Step 1: Learn About Neighbors. When a router is booted, it sends a special HELLO packet along each line; the router at the other end sends a reply saying who it is.

Step 2: Measuring Line Cost. Each router must estimate its delay to its neighbors. One approach: Send an ECHO packet to each neighbor which is time-stamped and sent back.

Step 3: Build Link State Packets. Each packet is labeled by: (1) the router that builds it; (2) a sequence number; (3) an age--which is decremented as it is transferred; (4) the router's neighbors and the line costs.

Step 4: Distribute Link State Packets. The packets are sent to all other routers by flooding; sequence numbers are used so that duplicates and older versions can be killed. Likewise, it is given an age which is decremented regularly; if it hits zero, the packet is killed.

Step 5: Compute New Routes. Since complete information is available on all costs, each router can now compute a route using, for example, Dijkstra's algorithm.



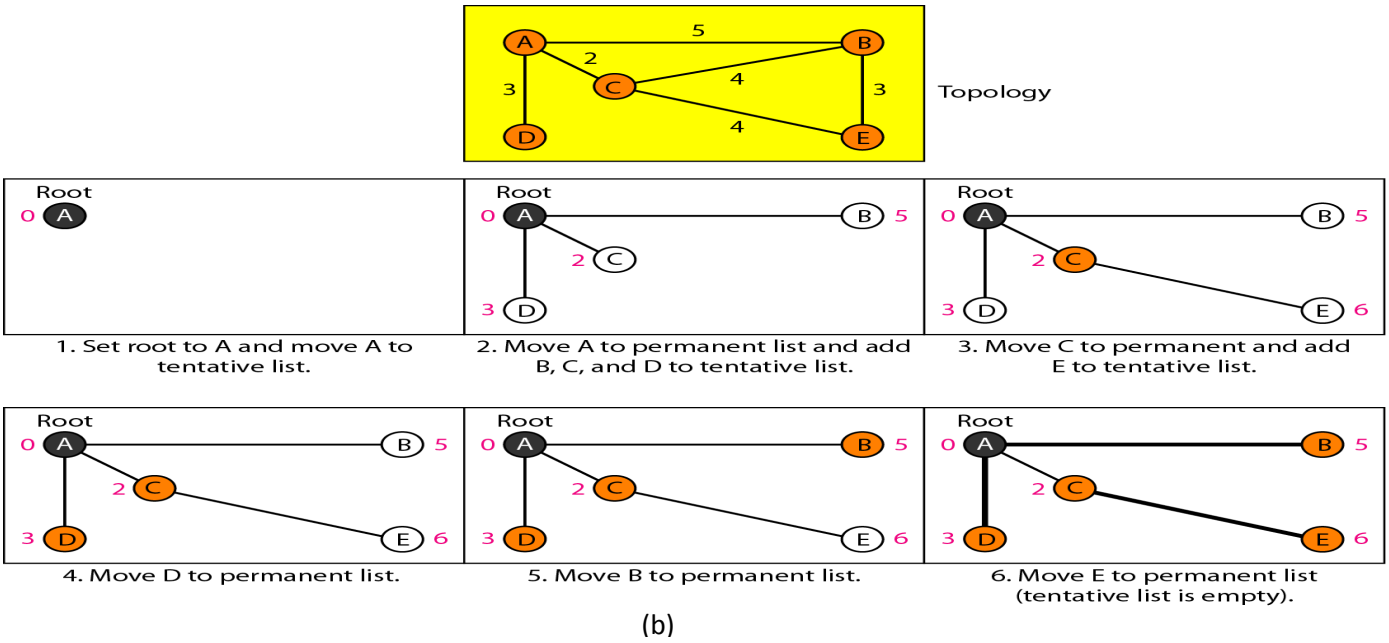


Figure (a) A network containing 5 nodes and (b) Creating new routes using Dijkstra's algorithm